

CSCE 735 PARALLEL COMPUTING · MODULE 07 · LECTURE NOTES

Performance Analysis

Finding out what is actually slow.

MODULE OUTCOMES

- Build and read a roofline model.
- Compute arithmetic intensity for a kernel.
- Determine whether a kernel is compute- or bandwidth-bound.
- Use a profiler to identify the limiting resource.
- Measure and report performance honestly.

1 The roofline model

$$I = \text{FLOPs} / \text{bytes} \quad P = \min(P_{\text{peak}}, I \times BW_{\text{peak}}) \quad I_{\text{ridge}} = P_{\text{peak}} / BW_{\text{peak}}$$

Arithmetic intensity I is the number of floating-point operations performed per byte moved between memory and the processor. **It is a property of the algorithm and its data layout, not of the machine.** **Peak performance** and **peak bandwidth** are properties of the machine. The model says attainable performance is whichever of the two limits binds first.

Kernel	Arithmetic intensity	Consequence
Vector add: $c[i] = a[i] + b[i]$	1 FLOP per 12 bytes ≈ 0.08	Severely bandwidth-bound. The processor performs one addition while waiting for twelve bytes.
SAXPY: $y[i] = a \cdot x[i] + y[i]$	2 FLOP / 12 bytes ≈ 0.17	Bandwidth-bound.
7-point stencil	≈ 0.5 , with good reuse	Bandwidth-bound, and improvable by blocking.
Matrix multiply, naive	≈ 0.25	Bandwidth-bound — which surprises people, since matrix multiply is the canonical compute-heavy kernel.
Matrix multiply, tiled	$O(\text{tile dimension})$	Compute-bound. Each tile loaded once serves many operations, so intensity rises with the tile size. This is the entire reason tiling is the canonical optimisation , and it is the clearest demonstration that intensity is algorithmic rather than fixed.
FFT	$O(\log n)$	Depends on the transform size and the cache.

MOST REAL CODE IS BANDWIDTH-BOUND

Typical ridge points on modern hardware are 5 to 20 FLOP per byte, and they have been rising for decades — compute capacity has grown considerably faster than memory bandwidth, so the ridge moves right and more kernels fall on the bandwidth side.

Most real kernels have arithmetic intensities well below 1, as the table above shows. They are nowhere near the ridge.

So optimising the arithmetic in a bandwidth-bound loop achieves nothing at all. Making the floating-point operations faster, or fewer, does not help when the processor is idle waiting for data — and a great deal of optimisation effort is spent here.

What does help: raise the arithmetic intensity through blocking and loop fusion, so that each byte fetched is used more times; **reduce the bytes moved** through better data layout, smaller data types, and compression; and **improve locality** so that fetches hit in cache. **Not faster arithmetic.** The roofline exists to tell you this before you spend a week on the wrong thing.

2 Profiling

- 1 **Start with timers around phases.** Coarse, free, requires no tooling, and it usually identifies the right region immediately. A great deal of profiling effort is spent discovering something a `printf` of elapsed time would have revealed.
- 2 **Then a sampling profiler** — `perf`, VTune, Instruments, or your platform's equivalent. Low overhead, no instrumentation, and it tells you **where** the time goes.
- 3 **Then hardware performance counters** for **why**: cache misses at each level, branch mispredictions, memory stall cycles, vector instruction counts, coherence traffic.
- 4 **Measure the achieved bandwidth and FLOP rate**, and place the kernel on the roofline. This tells you which ceiling you are near and therefore which optimisations can possibly help.
- 5 **And only then optimise** — against a measurement rather than an intuition about what ought to be slow. **The ordering matters**, and people routinely reach for hardware counters before they know which function is responsible.

Counter symptom	Likely cause	What to do
High cache miss rate	Poor locality, or a data layout that fetches unused bytes.	Blocking, structure-of-arrays, smaller data types (Module 02).
High branch misprediction rate	Data-dependent branches that the predictor cannot learn.	Branchless formulations, sorting the data, or predication.
Low instructions-per-cycle with high memory stalls	Bandwidth- or latency-bound.	Place it on the roofline; raise intensity or reduce bytes.
Low vector instruction fraction	The loop is not vectorising.	Read the vectorisation report (Module 02).

Counter symptom	Likely cause	What to do
High coherence or snoop traffic	False sharing , or genuine heavy sharing.	Pad to cache lines; use per-thread accumulators (Module 02).
Uneven cycle counts across threads	Load imbalance.	Finer granularity or dynamic scheduling (Module 06).

3 Measuring and reporting honestly

THE MEASUREMENT MISTAKES THAT RECUR

No warm-up. The first iteration pays for cold caches, page faults on first touch, lazy allocation, and in a JIT-compiled language the compilation itself. Discard it, or run long enough that it does not matter.

Dead code elimination. If a benchmark computes a result nothing uses, the compiler may remove the computation entirely and you measure an empty loop — which frequently appears as an implausibly excellent result. Consume the result, or use a compiler-specific `DoNotOptimize`.

Timing a single run. The machine has other work, the scheduler moves threads, and frequency varies.

Report a distribution — median and spread over many runs — rather than one number or a minimum.

Measuring at the wrong granularity. A timer call around an operation taking a microsecond measures mostly the timer. Loop the operation and divide.

Frequency scaling. Turbo makes the first seconds faster than the rest; thermal throttling makes the later minutes slower. **A benchmark of a few seconds and one of a few minutes measure different machines,** and which you want depends on the real workload.

- **The hardware:** processor model, core count, memory configuration and speed, and for GPU work the exact device. **'A modern CPU' is not a specification** and makes a result uncheckable.
- **The compiler and its flags.** **A factor of two frequently lives here,** and a comparison between two implementations built with different flags is not a comparison of the implementations.
- **The baseline and its optimisation state** (Module 01 §3). This is the one most often omitted and most often decisive.
- **The distribution rather than the minimum** — median, and some measure of spread, over enough runs to be meaningful.
- **The problem size,** because performance frequently depends on whether the working set fits in cache, and a result at one size may reverse at another. **Projects 1 and 2 require all five,** and a result lacking them cannot be checked or reproduced.

OPTIMISE IN THE RIGHT ORDER

- 1. A better algorithm.** An $O(n \log n)$ method beats any amount of tuning applied to an $O(n^2)$ one, at sufficient scale, and the crossover is usually smaller than people expect. CSCE 629's content is the highest-leverage optimisation available.
- 2. Better data layout.** Structure-of-arrays, blocking for cache, alignment, smaller types. **Frequently a larger factor than parallelism**, and it compounds with everything after it — a bandwidth-bound kernel parallelises badly, and fixing the layout fixes the parallel version too.
- 3. Vectorisation.** Four to sixteen times on a single core, frequently for the cost of a compiler flag and a `__restrict__` (Module 02).
- 4. Parallelism.** Bounded above by Amdahl's law and, for most real kernels, by the memory system rather than the core count.
- 5. Micro-optimisation.** Last, and usually unnecessary once the first four are done. **Performing these in the wrong order wastes most of the effort** — hand-tuning the inner loop of an algorithm you are about to replace is the characteristic failure.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Williams, Waterman & Patterson — Roofline: An Insightful Visual Performance Model (free)	The roofline paper. Short, and the model has held up remarkably well. Read it before building your first roofline.
Brendan Gregg — systems performance resources (free)	Profiling methodology, flame graphs, and the USE method. The practical side of §2, and exhaustively documented.
Hoefler & Belli — Scientific Benchmarking of Parallel Computing Systems (free)	The reporting discipline of §3, made rigorous. Read it before producing any performance claim.
Intel — VTune and Advisor documentation (free)	Advisor generates roofline plots automatically, which is a considerable convenience.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure your machine's peak FLOP rate and peak memory bandwidth, and compute the ridge point.

- 2 Compute the arithmetic intensity of five kernels by hand and verify each against measured bytes and FLOPs.
- 3 **Build a roofline plot for your machine** and place those five kernels on it.
- 4 Implement naive and tiled matrix multiply. Measure the intensity and the performance of each and place both on the roofline.
- 5 Sweep the tile size and plot performance against it. Explain the shape.
- 6 Take a bandwidth-bound kernel and optimise its arithmetic. Confirm nothing happens.
- 7 Then reduce its bytes moved — by changing layout or data type — and measure again.
- 8 Profile a program with timers, then a sampling profiler, then hardware counters. Note what each step added.
- 9 **Construct a benchmark that the compiler optimises away** and demonstrate the implausible result. Then fix it.
- 10 Run a benchmark for 1 second and for 5 minutes on the same machine and compare, identifying frequency scaling effects.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define arithmetic intensity and say what it is a property of.
- 2 State the roofline model and explain the ridge point.
- 3 Why is naive matrix multiply bandwidth-bound and tiled matrix multiply not?
- 4 Why is most real code bandwidth-bound, and what follows for optimisation?
- 5 Give the four profiling steps in order and say what each provides.
- 6 Give six counter symptoms and their likely causes.
- 7 Name five recurring measurement mistakes.
- 8 What five things must a performance claim state?
- 9 Give the five optimisation steps in order and say what goes wrong if they are reordered.